

BUỔI 11

Thanh toán & Cập nhật công nợ

Mỗi lần thu tiền → 1 Payment record → recalc Invoice

MỤC TIÊU BUỔI HỌC

- Tách Payment entity riêng khỏi Invoice
- Cập nhật amountPaid và balanceAmount sau mỗi lần thu
- Hỗ trợ partial payment nhiều lần

1

Mở File

Xem đúng file lỗi
trước khi code

2

Workflow

Hiểu bài toán &
luồng dữ liệu

3

Demo API

Chạy request theo
thứ tự trên Postman

4

Bài tập

Mở rộng từ demo
sinh viên tự làm

CÂU HỎI DẪN NHẬP

- Nếu chỉ lưu amountPaid trong Invoice thì ta mất gì?
- Payment có nên xóa cứng (hard delete) không? Tại sao?

1

Payment.java

→ Lưu lịch sử từng lần thu tiền

2

PaymentCreateRequest.java

→ Input DTO cho thu ngân nhập liệu

3

PaymentResponse.java

→ Kết quả trả về sau khi ghi nhận thanh toán

4

BillingService.java

→ Method addPayment + recalculate invoice

5

BillingController.java

→ Endpoint POST /payments

1

Thu ngân chọn invoice cần thanh toán

Tìm theo invoiceId, kiểm tra status còn nợ không

2

Nhập số tiền và phương thức thanh toán

PaymentMethod: CASH / BANK_TRANSFER / CARD

3

Tạo record Payment mới

Lưu vào DB — đây là lịch sử không xóa

4

Cập nhật amountPaid + balanceAmount

$\text{SUM}(\text{payments}) \rightarrow \text{amountPaid}$; $\text{balance} = \text{final} - \text{paid}$

5

Đổi status invoice tương ứng

$0 < \text{paid} < \text{final} \rightarrow \text{PARTIALLY_PAID}$; $\text{paid} = \text{final} \rightarrow \text{PAID}$

LƯU Ý QUAN TRỌNG

 Mỗi lần thu tiền đều để lại lịch sử.

Invoice chỉ giữ con số tổng hợp sau khi recalc.

Không bao giờ xóa Payment record — đây là audit trail tài chính.

1

Invoice = 2.000.000 VNĐ

finalAmount = 2,000,000 | balanceAmount = 2,000,000 | UNPAID

2

Lần 1: đóng 800.000

amountPaid = 800,000 | balance = 1,200,000

3

→ Status: PARTIALLY_PAID

$0 < \text{amountPaid} < \text{finalAmount}$

4

Lần 2: đóng thêm 1.200.000

amountPaid = 2,000,000 | balance = 0

5

→ Status: PAID ✓

amountPaid $\geq$ finalAmount → đóng đủ học phí

LƯU Ý QUAN TRỌNG

 Lợi ích của partial payment:

- Phụ huynh linh hoạt đóng nhiều lần
- Trường vẫn theo dõi được số còn nợ
- Lịch sử minh bạch cho cả hai bên

ENTITY / ENUM

Mô tả cấu trúc dữ liệu

- **Payment**
- **PaymentMethod**
- **TuitionInvoice**

SERVICE

Giữ toàn bộ nghiệp vụ

- **BillingService**

CONTROLLER

Mở REST API ra ngoài

- **BillingController**

💡 Sau mỗi lần gọi addPayment → BillingService PHẢI recalculate lại amountPaid và balanceAmount của Invoice đó.

POST

/api/billing/payments

Ghi nhận 1 lần thanh toán, recalc invoice

GET

/api/billing/invoices/{invoiceId}/payments

Lịch sử thanh toán của 1 invoice

▶ TRÌNH TỰ DEMO

- ① Chạy request thành công một lần đầu
- ② Sửa dữ liệu để thấy điểm thất bại
- ③ Xem kết quả trong DB và response DTO

👁 QUAN SÁT KHI DEMO

- ① invoiceId trong request body hợp lệ chưa?
- ② Method recalcInvoice được gọi đúng chỗ không?
- ③ Response có trả đủ status mới của invoice không?



NHỮNG ĐIỀU BẮT BUỘC PHẢI ĐÚNG



Tách payment history ra entity riêng — không nhét vào Invoice



Sau mỗi Payment → bắt buộc recalculate lại Invoice



PaymentMethod phải là enum rõ ràng (CASH / BANK_TRANSFER / CARD)



Không để amountPaid vượt finalAmount nếu không có logic overpayment



BÀI TẬP

1. Tạo 1 invoice và thực hiện 2 lần thanh toán (partial payment)
2. Hiển thị lịch sử thanh toán chi tiết cho 1 invoice
3. Bổ sung validation: số tiền thanh toán phải > 0 và $\leq \text{balanceAmount}$



SAU BUỔI NÀY SINH VIÊN CÓ THỂ:

- ✓ Thu học phí nhiều lần và cập nhật đúng trạng thái
- ✓ Hiểu vòng đời một Invoice từ UNPAID → PAID
- ✓ Biết file lỗi nào để tự học tiếp



THÔNG ĐIỆP CHỐT BUỔI

"Hiểu workflow trước, đọc đúng file trước, rồi mới code nhanh và đúng."